

Gli algoritmi quadratici non sono sempre più lenti di quelli logaritmici ma esiste una soglia crossover al di sotto della quale conviene utilizzare algoritmi $O(n^2)$.

Con lo scopo di trovare tale soglia, ho confrontato i tempi di esecuzione degli algoritmi di ordinamento effettuando misurazioni su vettori di dimensioni ridotte n (da 4 a 100).

Dai dati ottenuti, si è osservato che l'algoritmo quadratico più veloce è l'Insertion Sort: per $n=10$, ad esempio, impiega 1.25×10^{-7} s, contro i 1.93×10^{-7} s del Quick Sort e circa 1.18×10^{-6} s del Merge Sort.

Tra gli algoritmi $O(n \log n)$, il più efficiente risulta essere il QuickSort. MergeSort presenta tempi significativamente più elevati per vettori di piccole dimensioni.

Per tale motivo, per trovare la soglia, mi è bastato confrontare Insertion Sort e QuickSort. Tuttavia, il valore trovato oscilla indicativamente tra 10 e 40, in quanto strettamente dipendente anche dalla modalità di compilazione.

- **Modalità Debug:** compilando in Debug, la soglia è risultata elevata (circa 40). Questo accade perchè non viene applicata nessuna ottimizzazione: il compilatore non cerca di rendere il codice più veloce e non viene eliminata la ricorsione, per cui ogni chiamata del Quicksort richiede molto tempo. L'insertion sort, che invece è costituito da un ciclo semplice senza ricorsione, ne risente molto meno.
- In **Modalità Release**, ovvero con ottimizzazioni, la soglia è scesa anche fino a $n=10$.

Dopo aver trovato, indicativamente, il valore soglia, ho provato ad implementare una versione modificata del QuickSort che ha mostrato tempi inferiori rispetto al QuickSort puro. La versione ibrida, infatti, al di sotto della soglia, utilizza l'Insertion Sort, che abbiamo visto essere molto più veloce per dimensioni ridotte.

Infine, l'algoritmo della libreria standard (`std::sort`) è risultato quasi sempre il più veloce, mantenendo tempi quasi identici al Quick Sort ibrido. Questo è coerente col fatto che anche `std::sort` utilizza un algoritmo ibrido.

